

Министерство образования Республики Беларусь
Учреждение образования
“Брестский государственный технический университет”
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчёт по лабораторной работе № 5
По дисциплине «Современные платформы программирования»

Выполнил:
Тютюков К. О.
Студент группы ПО-13
Проверил:
А.А. Крощенко
Доцент кафедры ИИТ
03.04.2026

Цель работы: приобрести практические навыки разработки API и баз данных

Ход работы:

Общее задание

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт; Базу данные можно реализовать в любой СУБД (MySQL, PostgreSQL, SQLite и др.)

20) База данных Справочник товаров и услуг

database.py:

```
from sqlalchemy import create_engine, Column, Integer, String, Float, ForeignKey
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker, relationship
```

```
Base = declarative_base()
```

```
class Category(Base):
    __tablename__ = "categories"
    id = Column(Integer, primary_key=True)
    name = Column(String(100), nullable=False)
```

```
class Supplier(Base):
    __tablename__ = "suppliers"
    id = Column(Integer, primary_key=True)
    name = Column(String(200), nullable=False)
    phone = Column(String(20))
    address = Column(String(500))
```

```
class Product(Base):
```

```
__tablename__ = "products"
id = Column(Integer, primary_key=True)
name = Column(String(200), nullable=False)
price = Column(Float, nullable=False)
stock = Column(Integer, default=0)
category_id = Column(Integer, ForeignKey("categories.id"))
supplier_id = Column(Integer, ForeignKey("suppliers.id"))
```

```
class Service(Base):
    __tablename__ = "services"
    id = Column(Integer, primary_key=True)
    name = Column(String(200), nullable=False)
    price = Column(Float, nullable=False)
    duration = Column(Integer, default=60)
    category_id = Column(Integer, ForeignKey("categories.id"))
```

```
class Order(Base):
    __tablename__ = "orders"
    id = Column(Integer, primary_key=True)
    total = Column(Float, default=0)
    status = Column(String(50), default="новый")
    customer_name = Column(String(200))
```

```
class OrderItem(Base):
    __tablename__ = "order_items"
    id = Column(Integer, primary_key=True)
    order_id = Column(Integer, ForeignKey("orders.id"))
    product_id = Column(Integer, ForeignKey("products.id"))
    quantity = Column(Integer, nullable=False)
    price = Column(Float, nullable=False)
```

```
engine = create_engine("sqlite:///catalog.db", connect_args={"check_same_thread": False})
Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
```

```
def init_db():
    db = Session()
```

```
    if db.query(Category).count() == 0:
        db.add_all([Category(name="Электроника"), Category(name="Мебель"),
                    Category(name="Услуги")])
    db.commit()
```

```

if db.query(Supplier).count() == 0:
    db.add_all(
        [
            Supplier(name="ООО Поставка", phone="8-800-123-45-67", address="Москва"),
            Supplier(name="ИП Иванов", phone="8-912-345-67-89", address="СПб"),
        ]
    )
    db.commit()

```

```

if db.query(Product).count() == 0:
    db.add_all(
        [
            Product(name="Ноутбук", price=50000, stock=10, category_id=1, supplier_id=1),
            Product(name="Телефон", price=30000, stock=20, category_id=1, supplier_id=1),
            Product(name="Стул", price=5000, stock=15, category_id=2, supplier_id=2),
            Product(name="Стол", price=10000, stock=5, category_id=2, supplier_id=2),
        ]
    )
    db.commit()

```

```

if db.query(Service).count() == 0:
    db.add_all(
        [
            Service(name="Ремонт", price=2000, duration=60, category_id=3),
            Service(name="Доставка", price=500, duration=30, category_id=3),
        ]
    )
    db.commit()

```

```

db.close()

```

```

init_db()

```

main.py:

```

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import Optional, List
from database import Session, Category, Supplier, Product, Service, Order, OrderItem

```

```

app = FastAPI(title="Справочник товаров и услуг")

```

```

class CategoryCreate(BaseModel):

```

```
name: str
```

```
class SupplierCreate(BaseModel):  
    name: str  
    phone: str  
    address: str
```

```
class ProductCreate(BaseModel):  
    name: str  
    price: float  
    stock: int  
    category_id: int  
    supplier_id: int
```

```
@app.get("/")  
def root():  
    return {"message": "Справочник товаров и услуг"}
```

```
@app.get("/categories")  
def get_categories():  
    db = Session()  
    result = db.query(Category).all()  
    db.close()  
    return result
```

```
@app.post("/categories")  
def create_category(cat: CategoryCreate):  
    db = Session()  
    new = Category(name=cat.name)  
    db.add(new)  
    db.commit()  
    db.refresh(new)  
    db.close()  
    return new
```

```
@app.get("/suppliers")  
def get_suppliers():  
    db = Session()  
    result = db.query(Supplier).all()  
    db.close()  
    return result
```

```
@app.post("/suppliers")
def create_supplier(sup: SupplierCreate):
    db = Session()
    new = Supplier(name=sup.name, phone=sup.phone, address=sup.address)
    db.add(new)
    db.commit()
    db.refresh(new)
    db.close()
    return new
```

```
@app.get("/products")
def get_products():
    db = Session()
    result = db.query(Product).all()
    db.close()
    return result
```

```
@app.post("/products")
def create_product(prod: ProductCreate):
    db = Session()
    new = Product(
        name=prod.name, price=prod.price, stock=prod.stock, category_id=prod.category_id,
        supplier_id=prod.supplier_id
    )
    db.add(new)
    db.commit()
    db.refresh(new)
    db.close()
    return new
```

```
@app.put("/products/{id}")
def update_product(id: int, prod: ProductCreate):
    db = Session()
    p = db.query(Product).filter(Product.id == id).first()
    if not p:
        raise HTTPException(404, "Товар не найден")
    p.name = prod.name
    p.price = prod.price
    p.stock = prod.stock
    p.category_id = prod.category_id
    p.supplier_id = prod.supplier_id
    db.commit()
    db.close()
    return p
```

```
@app.delete("/products/{id}")
def delete_product(id: int):
    db = Session()
    p = db.query(Product).filter(Product.id == id).first()
    if not p:
        raise HTTPException(404, "Товар не найден")
    db.delete(p)
    db.commit()
    db.close()
    return {"message": "Товар удален"}
```

```
@app.get("/services")
def get_services():
    db = Session()
    result = db.query(Service).all()
    db.close()
    return result
```

```
@app.get("/orders")
def get_orders():
    db = Session()
    result = db.query(Order).all()
    db.close()
    return result
```

```
if __name__ == "__main__":
    import uvicorn
```

```
uvicorn.run(app, host="127.0.0.1", port=8000)
```

Справочник товаров и услуг

0.1.0

OAS3

/openapi.json

default

GET	/	Root
GET	/categories	Get Categories
POST	/categories	Create Category
GET	/suppliers	Get Suppliers
POST	/suppliers	Create Supplier
GET	/products	Get Products
POST	/products	Create Product
PUT	/products/{id}	Update Product
DELETE	/products/{id}	Delete Product

Вывод работы: приобрёл практические навыки разработки API и баз данных